Use Case Model: Sudoku Solver System

System Boundary

- **System:** Sudoku Solver Application (C++ program)
- **Primary Actor: User / System Operator** (initiates the program execution to process puzzle files)
- **Secondary Actor: File Storage System** (provides input text files `puzzle1.txt` through `puzzle5.txt`)

a. Identify/Discover a Use Case

Based on the functional requirements of the C++ Sudoku solver program, the core use case discovered is:

- **`Solve Sudoku Puzzles`**

b. Provide a Brief Description for the Use Case

- **Description:** This use case allows a User to automatically load, parse, solve, and display partially completed Sudoku puzzles from text files (`puzzle1.txt` through `puzzle5.txt`) using a depth-first search with backtracking algorithm. The system displays the file name, the original puzzle under a `Puzzle:` header, all valid solutions found (separated by a blank line), and the total count of solutions.

c. Provide a Step-by-Step Flow (Outline) of the Events for the Use Case

Following the standard outline structure:

1. User executes the Sudoku solver program.
2. System initializes the batch processing loop for puzzle files 1 through 5.
3. System reads and loads a puzzle file from storage.

4. System parses character data and populates the `SudokuBoard` grid.

5. System displays the puzzle file name and original puzzle grid.

6. System executes recursive depth-first search with backtracking to find all valid solutions.

7. System outputs the solved grid(s) and total solutions found.

d. Provide a Detailed Description for the Use Case

Structured into round-trip steps with complete descriptions:

1. Load Puzzle Files

- **User Action:** The user starts the C++ application executable.

- **System Response:** The system loops through file identifiers from `puzzle1.txt` to `puzzle5.txt`. For each iteration, it attempts to open the text file via an input file stream (`std::ifstream`). If a file cannot be opened, an error message is printed. If successful, it reads 81 characters representing the 9x9 grid, mapping underscores (_) to blank cells (`0`) and valid digit characters (1-9) to their integer equivalents.

2. Display Original Puzzle

- **User Action:** None (system-driven sequence).

- **System Response:** The system prints the current puzzle file name followed by the `Puzzle:` header label. It iterates through the internal 9x9 grid data structure and prints each cell to standard output, rendering blank cells (`0`) as underscores (_) and numbers with appropriate spacing, concluding with a blank separator line.

3. Solve Puzzle via Backtracking

- **User Action:** None.

- **System Response:** The system invokes the recursive solver function (`solveRecursive`). It searches for the next empty cell on the board. If no empty cells remain, a valid solution configuration is successfully captured and stored in a vector

collection. If empty cells exist, the system iterates through candidate digits 1 through 9, validates each candidate against row, column, and 3x3 subgrid constraints (`isValid`), places valid digits temporarily, recursively processes subsequent cells, and backtracks by resetting cells to `0` when paths fail.

4. Display Solutions and Summary

- **User Action:** None.

- **System Response:** The system evaluates the total number of solutions captured. If the solution count is zero, it prints `No solution found`. Otherwise, it prints every discovered solution grid row-by-row with single-space formatting, inserting a blank space between multiple solutions if more than one exists. Finally, it prints the summary line `Solutions found: [count]` and proceeds to the next puzzle file.